

PA6_Theory

1.

Stack:

```
factorial(1)
return -> factorial(2) call
dynamic -> factorial(2)
static -> factorial(2)
```

```
factorial(2)
return -> factorial(3) call
dynamic -> factorial(3)
static -> factorial(3)
```

```
factorial(3)
return -> factorial(4) call
dynamic -> factorial(4)
static -> factorial(4)
```

```
factorial(4)
return -> original call
dynamic -> none
static -> none
```

Unwind:

```
factorial(1) returns 1
|
factorial(1) is popped
|
factorial(2) returns 2 × 1 = 2
|
factorial(2) is popped
|
factorial(3) returns 3 × 2 = 6
|
factorial(3) is popped
|
factorial(4) returns 4 × 6 = 24
|
factorial(4) is popped
```

2.

```
def A() {
  def B() {
    return "Hi :)";
  }
  C();
}
```

```
def C() {  
    B();  
}
```

Static link -> A
Dynamic link -> C

Static links are still important because they show where in the program a function was defined, which can help find variables, while dynamic links only show which function called it.

3. $n \leq S/R$

Tail-recursive functions do not avoid this limit in a naive tree-walking interpreter because a new activation record is still created for each recursive call. Even though the recursive call is last, the previous frames still exist. A production compiler could optimize this by reusing the same frame, but a naive interpreter does not.